

Data Cleaning Workflow for the Reconciled Formula 1 Database

Giuseppe Rudi

May 15, 2026

Introduction

This document describes the Data Cleaning workflow adopted for the Formula 1 Data Warehouse project.

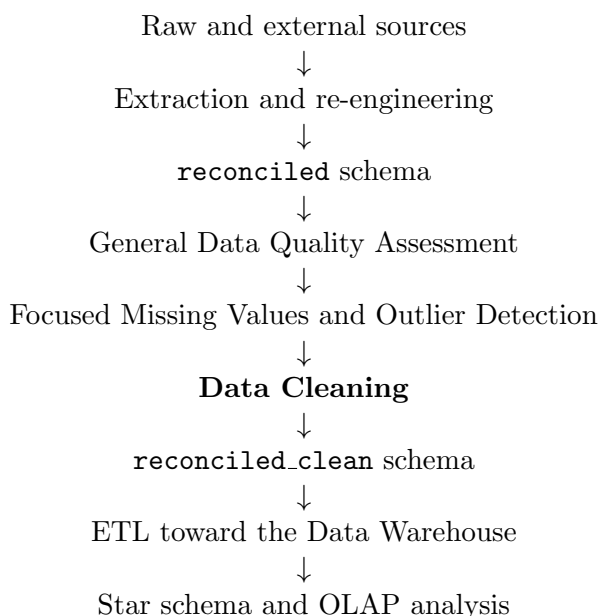
The goal of the Data Cleaning phase is not to blindly modify every value that is reported as problematic. Instead, the goal is to apply a small set of explicit, deterministic, and documented cleaning policies.

In this project, the Data Quality Assessment identifies possible data quality issues. The Missing Values script interprets missing values in the most important analytical tables. The Outlier Detection script flags suspicious numerical values. The Data Cleaning phase uses these outputs as evidence and decides what should be done with the affected rows or values.

The cleaning approach is conservative. Values are modified only when the correction is deterministic. Rows are removed only when they are structurally unusable or cannot support the analytical grain required by the later Data Warehouse. In many cases, the row is preserved and enriched with quality flags.

Position in the Project Pipeline

The Data Cleaning phase is placed after the quality assessment steps and before the ETL toward the Data Warehouse.



The original reconciled data are not overwritten. The cleaned output is written into a separate schema named:

reconciled_clean

This separation makes the cleaning phase reproducible and inspectable. It also allows a before/after comparison between the pre-cleaning and post-cleaning versions of the data.

Difference Between DQA, Focused Analysis, Cleaning, and Analytical Filtering

Several phases are involved in the preparation of the data. Each phase has a different role.

General Data Quality Assessment

The General Data Quality Assessment does not modify the database. It checks the reconciled tables according to general quality dimensions:

- completeness of required attributes;
- uniqueness of primary keys and natural keys;
- validity of numerical ranges and categorical domains;
- consistency between related attributes;
- referential integrity between tables.

Focused Missing Values and Outlier Detection

The focused scripts are applied only to the most important analytical tables:

- `lap`;
- `result`.

This phase is separated from the General DQA because missing values and outliers in Formula 1 data require domain interpretation.

For example, a missing `q3.ms` value may be expected if a driver did not reach Q3. Similarly, an extreme lap time may be caused by race context rather than by a data error.

Data Cleaning

Data Cleaning uses the outputs of the previous phases and applies documented policies.

The cleaning phase answers the following question:

Given a detected issue, what should be done with the affected row or value?

Some issues lead to standardization. Some issues lead to row exclusion. Some issues only lead to a quality flag.

Analytical Filtering

Analytical filtering is different from cleaning.

Data Cleaning handles data that are incomplete, inconsistent, invalid, duplicated, or structurally unusable. Analytical Filtering selects valid rows for a specific analysis.

Examples of analytical filtering are:

- keeping only dry sessions;
- selecting only Race sessions;
- comparing only selected teams;

Therefore, not every row excluded from a visualization is a bad row. It may simply be outside the scope of a specific analysis.

Input of the Data Cleaning Phase

The Data Cleaning phase uses three main categories of input.

Database Input

The script reads the pre-cleaning reconciled tables from:

`reconciled`

This schema contains the re-engineered tables before cleaning.

General DQA Outputs

The main input produced by the General DQA is:

`issues_all_tables.csv`

This file contains row-level issues detected by the General DQA.

The cleaning script uses the pair `check_id` and `issue_code` to select the corresponding cleaning policy.

Focused Missing Value Outputs

The main focused missing value output is:

`focused_missing_row_flags.csv`

This file contains one row for each relevant missing value detected in the `lap` and `result` tables.

The cleaning script does use `missing_information_area` to decide the action.

Focused Outlier Outputs

The main outlier detection output is:

`lap_outlier_flags.csv`

This file contains row-level outlier flags for selected numerical lap metrics.

Outlier values are not automatically considered wrong. They are used to add flags and, only in specific cases, to exclude rows from the later analytical fact.

Output of the Data Cleaning Phase

The Data Cleaning phase produces two types of output:

- cleaned database tables;
- audit and traceability files.

Cleaned Database Schema

The cleaned tables are written into:

`reconciled_clean`

The schema contains the cleaned version of the reconciled tables. For the most important analytical tables, additional quality columns may be added.

Audit Outputs

The cleaning phase also produces audit files.

Output File	Purpose
<code>cleaning_action_log.csv</code>	Records every cleaning action, standardization, flag or exclusion.
<code>cleaning_summary_by_table.csv</code>	Aggregates the number of actions applied to each table.
<code>cleaning_summary_by_policy.csv</code>	Aggregates the number of actions applied by cleaning policy.
<code>rejected_rows.csv</code>	Stores rows that are removed from the cleaned schema because they are structurally unusable.
<code>before_after_scorecard.csv</code>	Compares DQA scores and issue counts before and after cleaning.

General Cleaning Rules

General cleaning rules are applied before processing the DQA issue files. They are independent from the current DQA results.

In this project, general cleaning is intentionally simple. Since the dataset is obtained from structured APIs and loaded with explicit PostgreSQL types, the cleaning phase does not repeat generic parsing or typing rules.

The general rules are:

Rule ID	Condition	Action
CR_COMMON_1	Textual values contain leading or trailing spaces.	Trim textual values.
CR_COMMON_2	Empty strings are found in textual columns after trimming.	Convert empty strings into NULL.

These rules are considered safe because they do not change the semantic meaning of the data. They only standardize textual representation before applying more specific cleaning policies.

Cleaning Action Types

Each cleaning decision is associated with one action type.

In this project, the cleaning workflow uses four simple actions:

Action Type	Meaning
STANDARDIZE	The value is converted into a deterministic standard representation. This action is used only when the correction is safe and does not require guessing. Examples are trimming textual values, normalizing known categorical variants, or reconstructing a track status message from a documented status-code mapping.

Action Type	Meaning
SET_NULL	The value is replaced with NULL because it is invalid, suspicious, or not reliable, but the row itself can still be preserved. This is preferred over artificial placeholder values, because placeholders may pollute numerical calculations, categorical domains, and later OLAP analysis.
DROP_ROW	The row is removed from the cleaned schema. This action is used when the row is structurally unusable, when it cannot preserve the correct grain or relationships, or when a core analytical value is missing or strongly unreliable.
ADD_FLAG	The row is preserved, but a quality flag or missing information area is attached to it. This action is used when the row can still be useful for some analyses, but not for all analyses.

The cleaned database does not use artificial placeholder values such as -1, 9999, or textual values such as UNKNOWN inside typed analytical columns.

When a value is invalid and cannot be repaired deterministically, the preferred action is SET_NULL. This choice avoids introducing artificial values that could distort later statistical analysis, aggregations, or Data Warehouse measures.

For descriptive attributes that are not central to the analytical purpose, setting the invalid value to NULL is sufficient.

For analytically relevant attributes, instead, the script does not only set the invalid value to NULL. It also adds a corresponding flag that records which analytical information area has been affected.

For example, if an invalid sector time is found, the value is set to NULL and the row is marked with SECTOR_INFORMATION. This distinction is important because the cleaned schema must not hide the loss of analytical information. The NULL value removes the unreliable value, while the missing information flag explains which type of analysis may be affected. The audit log records why the value was changed to NULL.

DQA Actions by Table

The following tables summarize the direct connection between each DQA check and the cleaning action applied by the script.

In some cases, the action is conditional because the script first tries a deterministic standardization and drops or flags the row only if the value cannot be repaired safely.

Season Table

DQA Check	Action	Reason
season_required_structural_columns	DROP_ROW	The season cannot be identified without its structural key.
season_year_formula1_domain	DROP_ROW	season_year is the key of the table. If it is outside the Formula 1 domain, the row is not reliable.
number_of_events_positive	SET_NULL	The value is descriptive and not central for the analytical grain. An invalid value is removed instead of flagged.

DQA Check	Action	Reason
season_dates_order, season_dates_match_year	SET_NULL	Inconsistent season dates are not used as core analytical identifiers. They are set to NULL instead of using placeholders.

Circuit Table

DQA Check	Action	Reason
circuit_required_analytical_columns	DROP_ROW	The circuit classification is part of the analytical purpose of the table.
global_circuit_category_domain	STANDARDIZE / DROP_ROW	Known formatting variants are standardized. If the value cannot be mapped to the allowed domain, the row is dropped.
sector1_category_domain, sector2_category_domain, sector3_category_domain	STANDARDIZE / DROP_ROW	Sector categories are analytically important. Unknown values make the circuit classification unusable.

Driver Table

DQA Check	Action	Reason
driver_required_identification_columns	DROP_ROW	A driver without required identification data cannot be reliably referenced.
driver_abbreviation_format	SET_NULL	The abbreviation is useful for display, but an invalid abbreviation should not invalidate the whole driver row.
driver_url_format	SET_NULL	The URL is descriptive. If invalid, it is removed from the clean value.
permanent_number_range	SET_NULL	The permanent number is descriptive for this project. Invalid values are set to NULL.
driver_age_plausible_2021_2022	SET_NULL	If the derived age is implausible, the birth date is not trusted and is set to NULL.

Team Table

DQA Check	Action	Reason
team_required_identification_columns	DROP_ROW	A team without required identification data cannot be reliably referenced.
team_url_format	SET_NULL	The URL is descriptive and does not justify dropping the team row.

Grand Prix Table

DQA Check	Action	Reason
grand_prix_required_structural_columns	DROP_ROW	The Grand Prix cannot preserve its grain without required structural attributes.
round_number_positive	DROP_ROW	<code>round_number</code> is part of the natural key with <code>season_year</code> . If it is invalid, the event order and grain are unreliable.
event_format_domain	STANDARDIZE / DROP_ROW	Known variants can be standardized. Unknown event formats are not reliable for the project scope.
grand_prix_event_date_matches_season_year	SET_NULL	The date is useful but not the main identifier. If inconsistent with the season, it is set to NULL.

Session Table

DQA Check	Action	Reason
session_required_structural_columns	DROP_ROW	A session without structural identifiers cannot be used as a parent for results, laps, weather, or track status.
session_type_domain	STANDARDIZE / DROP_ROW	Known variants can be standardized. Unknown session types are outside the analytical scope.
session_date_near_grand_prix_date	SET_NULL	The session date is contextual. If it is not coherent with the Grand Prix date, it is removed from the cleaned value.

Result Table

DQA Check	Action	Reason
result_required_structural_columns	DROP_ROW	The result row cannot be linked to the correct session, driver, team, or position.
result_position_positive	DROP_ROW	<code>position</code> is classification information. If invalid, the result row cannot be used reliably.
result_points_non_negative	SET_NULL	Points are useful for some analyses, but invalid points do not invalidate the whole result row.
result_laps_non_negative	SET_NULL	Invalid lap count is removed from the cleaned value, but the classification row can remain usable.

DQA Check	Action	Reason
result_grid_position_non_negative	SET_NULL	Starting-grid information is contextual. Invalid values are set to NULL.

Lap Table

DQA Check	Action	Reason
lap_required_structural_columns	DROP_ROW	A lap without structural identifiers or timeline information cannot preserve the lap grain.
lap_number_positive	DROP_ROW	lap_number is part of the natural key. If invalid, the lap grain is unreliable.
lap_time_positive	DROP_ROW	lap_time_ms is the core lap performance measure. Invalid values make the lap unusable for the main fact.
lap_sector_times_positive	SET_NULL ADD_FLAG	+ Invalid sector values are removed, and the row is flagged as incomplete for sector analysis.
lap_speeds_positive	SET_NULL ADD_FLAG	+ Invalid speed values are removed, and the row is flagged as incomplete for speed analysis.
stint_positive, tyre_life_positive	SET_NULL ADD_FLAG	+ Invalid tyre-context values are removed, and tyre information is marked as incomplete.
compound_domain	STANDARDIZE / SET_NULL + ADD_FLAG	Known variants are standardized. Unknown compounds are set to NULL and flagged.
lap_sector_sum_matches_lap_time	ADD_FLAG	The row is preserved, but timing coherence is marked as suspicious.

Weather Table

DQA Check	Action	Reason
weather_required_columns	DROP_ROW	A weather observation without identifier, session, time, or core measures is not usable.
weather_time_non_negative	DROP_ROW	time_ms is part of the natural key with session.id. If invalid, the weather observation cannot be placed on the session timeline.
Weather range checks	SET_NULL	Invalid weather values are removed from the cleaned value. The weather row can still contain other useful observations.

DQA Check	Action	Reason
weather_track_air_temp_difference	SET_NULL	If the track-air temperature relation is not plausible, the less stable contextual value is removed.

Track Status Table

DQA Check	Action	Reason
track_status_required_columns	DROP_ROW	A track status record without status, message, time, or session context is not usable.
track_status_time_non_negative	DROP_ROW	time_ms is part of the natural key with session_id. If invalid, the record cannot be placed on the session timeline.
track_status_code_domain	DROP_ROW	Unknown status codes cannot be interpreted reliably.
track_status_message_domain	STANDARDIZE / SET_NULL	Known variants can be standardized. Unknown messages are set to NULL if the code remains valid.
track_status_message_mapping	STANDARDIZE	The message can be reconstructed deterministically from the status code.

Checks Applied to All Tables

DQA Check	Action	Reason
Primary key uniqueness checks	DROP_ROW	Rows with null or conflicting primary keys cannot preserve the table grain.
Natural key uniqueness checks	DROP_ROW	Conflicting duplicated natural keys make the row grain ambiguous.
Foreign key checks	DROP_ROW	Rows with broken references are removed because their relationships cannot be trusted.

Missing-value-driven Cleaning Actions

The focused missing value script produces row-level flags for the `lap` and `result` tables.

The cleaning script uses these flags directly. No additional policy identifier is needed.

For each focused missing value flag, the script checks:

```
table_name + row_identifier + column_name + missing_class + missing_information_area
```

Then it applies either `DROP_ROW` or `ADD_FLAG`.

However, in the cleaned schema, `ADD_FLAG` does not mean that all missing information areas are stored inside a single textual attribute.

The cleaned schema uses one boolean flag for each relevant information area.

The meaning of these flags is:

true means that the row has a missing or unreliable information problem in that analytical area.

For example:

- `has_tyre_information_issue = true` means that tyre information is incomplete or unreliable.

This representation is more suitable for SQL and OLAP queries. For example, to select laps that are usable for sector-based analysis, it is sufficient to filter:

`has_sector_information_issue = false`

Expected nulls with `missing_information_area = NONE` are not treated as cleaning problems. They are useful for documentation, but they do not require a transformation of the cleaned table.

Missing Value Actions for the Lap Table

For the `lap` table, the cleaning script creates a set of boolean quality flags.

Boolean Flag in Cleaned Table	Activated By	Meaning
<code>has_sector_information_issue</code>	<code>SECTOR_INFORMATION</code>	Sector information is missing or unreliable. The row should not be used for complete sector analysis.
<code>has_speed_information_issue</code>	<code>SPEED_INFORMATION</code>	Speed information is missing or unreliable. The row should not be used for speed-based analysis.
<code>has_tyre_information_issue</code>	<code>TYRE_INFORMATION</code>	Tyre information is missing or unreliable. The row should not be used for tyre-based comparison without caution.
<code>has_weather_information_issue</code>	<code>WEATHER_INFORMATION</code>	Weather context information is missing or unreliable. Weather-based comparison is limited.
<code>has_track_status_information_issue</code>	<code>TRACK_STATUS_INFORMATION</code>	Track status information is missing or not interpretable. Clean-lap filtering is limited.
<code>has_pit_information_issue</code>	<code>PIT_INFORMATION</code>	Pit-related interpretation is incomplete. Special-lap analysis is limited.

The information area `LAP_TIME_INFORMATION` is handled differently. If `lap_time_ms` is missing, the row is not flagged and preserved. Instead, the row is removed from the cleaned `lap` table, because `lap_time_ms` is the core measure of the Lap Performance fact.

Missing Information	Condition	Action	Reason
LAP_TIME_INFORMATION	lap_time_ms is missing	DROP_ROW	The row cannot support the main lap performance analysis without the lap time.
TYRE_INFORMATION	compound or tyre_life is missing	ADD_FLAG	The row is preserved and has_tyre_information_issue is set to true.
SECTOR_INFORMATION	One or more sector times are missing	ADD_FLAG	The row is preserved and has_sector_information_issue is set to true.
SPEED_INFORMATION	One or more speed values are missing	ADD_FLAG	The row is preserved and has_speed_information_issue is set to true.
TRACK_STATUS_INFORMATION	track_status is missing or not interpretable	ADD_FLAG	The row is preserved and has_track_status_information_issue is set to true.
WEATHER_INFORMATION	Lap-level weather context is missing	ADD_FLAG	The row is preserved and has_weather_information_issue is set to true.
PIT_INFORMATION	Pit interpretation is incomplete	ADD_FLAG	The row is preserved and has_pit_information_issue is set to true.
NONE	Expected null	No cleaning transformation	The missing value is expected and does not remove useful analytical information.

Missing Value Actions for the Result Table

For the **result** table, the same principle is used. The cleaned table does not store a combined textual list of missing information areas. Instead, it stores one boolean flag for each relevant result information area.

Boolean Flag in Cleaned Table	Activated By	Meaning
has_qualifying_information_issue	QUALIFYING_INFORMATION	Qualifying information is incomplete or suspicious. The row should not be used directly for complete qualifying time analysis.

Boolean Flag in Cleaned Table	Activated By	Meaning
<code>has_race_context_information</code>	<code>RACE_CONTEXT_INFORMATION</code>	Race context information is incomplete. The row may still be usable for classification analysis, but not for all race-context analyses.

The information area `CLASSIFICATION_INFORMATION` is handled more strictly. If position or classified position information is missing when required, the row is removed from the cleaned `result` table.

Missing Information	Condition	Action	Reason
<code>CLASSIFICATION_INFORMATION</code>	Position or classified position is missing when required	<code>DROP_ROW</code>	The result cannot support the Session Result fact without classification information.
<code>QUALIFYING_INFORMATION</code>	Required qualifying time is missing	<code>ADD_FLAG</code>	The row is preserved and <code>has_qualifying_information_issue</code> is set to true.
<code>RACE_CONTEXT_INFORMATION</code>	Race context attributes are missing	<code>ADD_FLAG</code>	The row is preserved and <code>has_race_context_information_issue</code> is set to true.
<code>NONE</code>	Expected null	No cleaning transformation	The missing value is expected in the current session context.

Outlier-driven Cleaning Actions

The focused outlier detection script is applied only to the `lap` table.

The script detects outliers for the following lap-level metrics:

Metric Group	Attributes
Lap and sector times	<code>lap_time_ms</code> , <code>sector1.time_ms</code> , <code>sector2.time_ms</code> , <code>sector3.time_ms</code>
Speed metrics	<code>speed_i1</code> , <code>speed_i2</code> , <code>speed_fl</code> , <code>speed_st</code>

The outlier detection output is not interpreted as automatic proof that a value is wrong. An outlier may be caused by real Formula 1 situations, by driver behaviour, by tyre degradation, or by source-specific recording effects.

For this reason, the cleaning script does not automatically replace outlier values with artificial values. The action depends on:

- the metric that was flagged;
- the consensus score returned by the outlier detection script.

The same quality-flag logic used for missing values is also used for outliers.

For example:

- a sector-time outlier activates `has_sector_information_issue`;
- a speed outlier activates `has_speed_information_issue`.

This means that the same flag can represent both missing information and unreliable information in the same analytical area. For example, `has_speed_information_issue = true` means that the speed information of the lap should not be trusted for speed-based analysis, either because it is missing or because it was detected as an outlier.

Metric	Consensus Score	Action	Reason
<code>lap_time_ms</code>	2	DROP_ROW	The lap time is the core measure of lap performance. If both statistical methods flag it, the row is considered too unreliable for the cleaned analytical database.
<code>lap_time_ms</code>	1	No cleaning transformation	A weak lap time anomaly is not enough to remove or flag the row during cleaning. The outlier report remains available for inspection.
<code>sector1_time_ms</code> , <code>sector2_time_ms</code> , <code>sector3_time_ms</code>	1 or 2	ADD_FLAG	The row is preserved, but <code>has_sector_information_issue</code> is set to true because sector information may be unreliable.
<code>speed_i1</code> , <code>speed_i2</code> , <code>speed_fl</code> , <code>speed_st</code>	1 or 2	ADD_FLAG	The row is preserved, but <code>has_speed_information_issue</code> is set to true because speed information may be unreliable.

This rule keeps the cleaning process simple.

A strong outlier in `lap_time_ms` removes the row because the main analytical measure is unreliable. In this case, adding a flag would not be useful, because the row would not be suitable for the main Lap Performance fact.

Outliers in sector or speed metrics do not remove the row. They only activate the corresponding boolean quality flag, because the same lap may still be useful for total lap time analysis.

For example, if `speed_i1` is detected as an outlier, the cleaned `lap` row can still be used for lap time analysis, but it should be excluded from speed-based visualizations by filtering:

```
has_speed_information_issue = false
```

Audit Log and Traceability

Every cleaning action must be documented.

The main audit output is:

```
cleaning_action_log.csv
```

The audit log records only the information needed to understand what triggered the cleaning action and what the script did.

After the introduction of boolean quality flags, the audit log does not store a combined textual list of missing information areas. Instead, when the cleaning action is **ADD_FLAG**, the log records the specific boolean flag column that was activated.

A possible structure is:

Column	Meaning
<code>source</code>	Origin of the action. Possible values are <code>general_cleaning</code> , <code>general_dqa</code> , <code>missing_values</code> , and <code>outlier_detection</code> .
<code>table_name</code>	Table affected by the cleaning action.
<code>row_identifier</code>	Identifier of the affected row. For example, <code>lap_id=...</code> or <code>result_id=...</code>
<code>target_column</code>	Column affected by the action when the action modifies or evaluates a specific source attribute. For example, <code>sector2.time_ms</code> , <code>speed_i1</code> , or <code>compound</code> .
<code>quality_flag_column</code>	Boolean quality flag activated by the cleaning action, when applicable. For example, <code>has_sector_information_issue</code> or <code>has_speed_information_issue</code> .
<code>check_id</code>	DQA check that generated the issue, when the action comes from the General DQA.
<code>issue_code</code>	Type of issue detected by the General DQA, when applicable.
<code>missing_class</code>	Missing value class, when the action comes from the focused missing value script. Possible values are <code>EXPLAINED_NULL</code> and <code>SUSPICIOUS_NULL</code> .
<code>information_area</code>	Analytical information area affected by the issue. This can come from missing values, DQA cleaning, or outlier detection. Examples are <code>SECTOR_INFORMATION</code> , <code>SPEED_INFORMATION</code> , <code>TYRE_INFORMATION</code> , and <code>QUALIFYING_INFORMATION</code> .
<code>outlier_metric</code>	Metric detected as an outlier, when the action comes from the outlier detection script.
<code>outlier_consensus_score</code>	Number of outlier methods that flagged the value. This is used only for outlier-driven cleaning actions.
<code>cleaning_action</code>	Action applied by the cleaning script. Possible values are <code>STANDARDIZE</code> , <code>SET_NULL</code> , <code>ADD_FLAG</code> , and <code>DROP_ROW</code> .
<code>old_value</code>	Value before cleaning, only when a value was modified. For <code>ADD_FLAG</code> , this can be the previous boolean value of the quality flag.
<code>new_value</code>	Value after cleaning, only when a value was modified. For <code>ADD_FLAG</code> , this is usually <code>true</code> .
<code>reason</code>	Short explanation of why the action was applied.

This audit log supports traceability and reproducibility.

The log does not contain a separate `policy_id` or `action_type`, because the cleaning decision is already represented directly by the column `cleaning_action`.

The column `information_area` explains the analytical meaning of the problem, while `quality_flag_column` explains which boolean flag was actually changed in the cleaned table.

For example, if `sector2_time_ms` is missing, the audit log records:

```
information_area = SECTOR_INFORMATION
```

and:

```
quality_flag_column = has_sector_information_issue
```

This makes the cleaning action clear, while avoiding the previous problem of storing multiple categories inside a single textual attribute.

Rejected Rows

Rows are physically removed from the cleaned schema only when they cannot be safely used or repaired.

Rejected rows are written to:

```
rejected_rows.csv
```

This file preserves the rejected data for inspection.

The cleaned schema contains only the rows that are considered usable after the cleaning rules have been applied. Rows with partial but still useful information are not rejected. Instead, they are preserved with additional flags, such as missing information areas or outlier indicators.

Validation After Cleaning

After the cleaning process, the General Data Quality Assessment should be executed again on the `reconciled_clean` schema.

This allows a comparison between:

- pre-cleaning issue counts;
- post-cleaning issue counts;
- pre-cleaning quality scores;
- post-cleaning quality scores.

The goal is not necessarily to obtain a perfect score for every table.

The goal is to show that the remaining issues are:

- expected;
- documented;
- not safely correctable;
- preserved with flags;
- outside the scope of automatic cleaning.

Conclusion

The Data Cleaning phase is designed as a controlled and documented transformation process.

The General DQA identifies structural and semantic issues. The focused Missing Values script explains missing values and identifies missing information areas. The Outlier Detection script flags suspicious numerical values. The Data Cleaning phase combines these outputs and applies explicit policies.